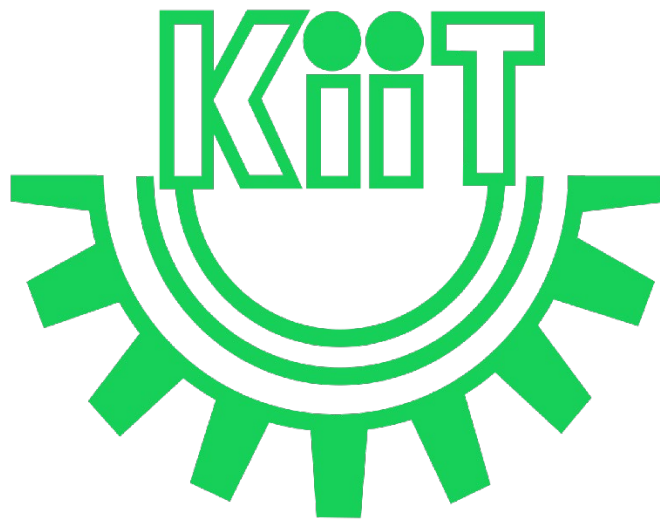


Integrated Eligibility System Using Spring boot, Microservices and MySQL

**Project submitted in partial fulfilment for the award of degree for
Master of Computer Application**

**Submitted By
Bailochan Padhi
Regd. No.: 22270988413**

**Under the Guidance of
Dr. Satya Ranjan Dash**



**SCHOOL OF COMPUTER APPLICATIONS
Kalinga Institute of Industrial Technology,
University, Bhubaneswar, Odisha
[April-2024]**



School of Computer Application

Kalinga Institute of Industrial Technology, Bhubaneswar, Odisha

Certificate

This is to certify that the project entitled **"Integrated Eligibility System Using Spring boot, Microservices and MySQL"** is being submitted by **Mr. Bailochan Padhi**, Regd. no. **22270988413**, to the Department of School of Computer Application, Kalinga Institute of Industrial Technology, Bhubaneswar, in partial fulfilment of the requirements for the degree of **Master Of Computer Application (MCA)** . It is an original report carried out by his under our supervision and guidance.

Head of Computer Application
KIIT Bhubaneswar

Dr. Satya Ranjan Dash
Supervisor



Certificate of Approval

This is to certify that the PROJECT REPORT entitled **"Integrated Eligibility System Using Spring boot, Microservices and MySQL"** submitted by **Mr. Bailochan Padhi** (Regd. no. **22270988413**) of **Master Of Computer Application (MCA)** Branch, Kalinga Institute of Industrial Technology , Bhubaneswar, Odisha has been examined by us.

Internal Examiner

External Examiner

Declaration

I declare that I am the sole author of this dissertation and that the work presented in it, unless otherwise referenced, is entirely my own. I also declare that the work has not been submitted, in whole or in part, to any other University as an exercise for a degree or any other qualification.

Bailochan Padhi
Regd. No.:22270988413

Acknowledgment

I would like to thank our supervisor **Dr. Satya Ranjan Dash** for giving us the opportunity to undertake this project. I am extremely grateful for his insightful guidance, timely advice, continuous support and motivation throughout our candidature. He helped me a lot to improve my both technical and presentation skills and he was very much supportive in every step of the execution of this project. He was also very much understanding about the internship situation and helped whenever required. I especially thank him for his kindness and excellent contributions. His timely remarks helped me in wrapping the major project in time. I'm deeply thankful to **Dr. Chinmaya Misra**, Department of School of Computer Application, for their moral support and providing us with all the necessary facilities during our candidature.

I would also like to thank my Team Leads Bijaya kumar mohanty who helped me to understand the concept of the backend of the webservices thoroughly using Java Spring boot. My friends Asutosh, Barsha and Bibhu helped to figure out the project ideation and gave constant feedback in improving the project. Moreover, they helped me in implementing different functionalities, improving features and shaping up my ideas during the development of this project.

Lastly, I would like to express my gratitude to my family and all my friends for their unconditional love and support. Without their encouragement, the journey would have been difficult.

Abstract

The Rhode Island Integrated Eligibility System (RI-IES) is a comprehensive application designed to streamline the process of providing health and insurance plans for citizens in need within the state of Rhode Island. The application offers various plans such as SNAP, CCAP, Medicaid, Medicare, and RIW, each tailored to specific eligibility criteria and demographic groups. Through a series of modules, including administration, user management, application registration, data collection, eligibility determination, correspondence, benefit issuance, and reporting, RI-IES efficiently manages the entire lifecycle of plan applications and approvals.

The system operates by collecting and analyzing citizen data to determine eligibility for the applied plans. Approved citizens receive benefits on a monthly basis, while those whose data does not meet eligibility criteria are denied access to the plans. The RI-IES application is exclusively for Rhode Island State citizens, ensuring targeted assistance to those in need within the state.

Key features include an administrative module for managing plans and case workers, a user module for registration, login, and dashboard access, as well as specialized modules for application registration, data collection, eligibility determination, correspondence management, benefit issuance, and reporting. Reports generated by RI-IES include plan-wise, status-wise, and monthly reports in Excel and PDF formats, providing comprehensive insights into program performance and citizen assistance metrics.

Overall, RI-IES represents a vital tool in the efficient and equitable distribution of health and insurance plans to Rhode Island State citizens, facilitating access to essential services for those in need while ensuring accountability and transparency in program administration.

Contents

Certificate	i
Declaration	iii
Acknowledgment	iv
Abstract.....	v
Contents	vi
List of Figures.....	vi
CHAPTER 1: Introduction	
1.1. Overview	1
1.2. Project Description.....	1
1.3. Purpose	2
1.4. Project Scope	2
1.5. Objectives.....	3
CHAPTER 2: Literature Review	
2.1. Motivations	4
2.2. Problem Description	5
CHAPTER 3: Preliminaries	
3.1. Methodologies Used	7
3.2. System Architecture.....	7
3.3. Technologies Used.....	9
3.4. Tools Used.....	12
CHAPTER 4: Methods & Implementation	
4.1. Actors	14
4.2. Life Cycle Model.....	14
4.3. Functionalities Requirements	14
4.4. Non Functionalities & Requirements	22
4.5. Priority Table.....	23
4.6. Entity Relationship Diagram	24
4.7. Database Schema Diagram	25
4.8. Data Flow Diagram.....	26
4.9. Code Snippet.....	27
CHAPTER 5: Results & Outputs.....	31
CHAPTER 6: Conclusion & Future Work	35

LIST OF FIGURES

Figure 4.6 : Entity Relationship Diagram.....	24
Figure 4.7 : Database Schema Diagram	25
Figure 4.8.1 : DFD Context Level.....	26
Figure 4.8.2 : DFD Level 1	26
Figure 4.9.1 : User Entity	27
Figure 4.9.2 : Plan Entity	28
Figure 4.9.3 : Plan Entity	29
Figure 4.9.4 : Plan Rest Controller.....	30
Figure 5.1 : Account Creation For Case Worker	31
Figure 5.2 : Email Generated For Case Worker	32
Figure 5.3 : Application Creation For User	33
Figure 5.4 : Eligibility Determination	34

CHAPTER 1

INTRODUCTION

1.1 OVERVIEW:

The RI-IES (Integrated Eligibility System) is a sophisticated application designed to address the health and insurance needs of Rhode Island State citizens who are facing financial constraints. The system encompasses a range of plans, including SNAP, CCAP, Medicaid, Medicare, and RIW, each customized to specific eligibility conditions. Through the application of cutting-edge Spring Boot microservices, the RI-IES aims to streamline the plan application process, determine citizen eligibility, and facilitate the issuance of benefits.

1.2 PROJECT DESCRIPTION:

The RI-IES is a comprehensive application developed to provide health and insurance plans for Rhode Island State citizens in need. The application offers the following plans on behalf of the government:

- 1 SNAP: Supplement Nutrition Assistance Program (Food Stamp Program)
- 2 CCAP: Child Care Assistance Program (Child Care Program)
- 3 Medicaid: Health Plan
- 4 Medicare: Health Plan for senior citizens (aged 65 and above)
- 5 RIW: Unemployment Plan (Rhode Island Works)

Each plan has specific eligibility criteria. Citizens in need can apply for these plans through the RI-IES application. Based on the eligibility rules of each plan and the data provided by the citizens, the RI-IES system determines the eligibility of citizens for the applied plans.

- Approved citizens receive benefits from the state government on a monthly basis.
- If a citizen's data matches the eligibility criteria of a plan, they are approved for the plan.
- If a citizen's data does not meet the eligibility criteria of a plan, they are denied access to the plan.

Note: The plans offered by the RI-IES application are exclusively for Rhode Island State Citizens.

Modules of IES:

- 1.1 Admin: Manages plans and case workers within the application.
- 1.2 User: Handles registration, login, forgot password, and provides a dashboard for users.
- 1.3 AR (Application Registration): Allows citizens to create applications for plans.
- 1.4 DC (Data Collection): Gathers necessary information for citizen plans.
- 1.5 ED (Eligibility Determination): Verifies citizen eligibility for the plans.
- 1.6 CO (Correspondence): Sends notices and communications to citizens.
- 1.7 BI (Benefit Issuance): Distributes benefit amounts to approved citizens.
- 1.8 Reports: Generates various reports (in Excel and PDF formats) including plan-wise, status-wise, and monthly reports.

1.3 PURPOSE:

The primary purpose of the RI-IES is to provide a centralized and efficient platform for managing health and insurance plans for citizens with financial constraints. By automating the application process, eligibility determination, and benefit issuance, the system aims to reduce administrative overhead and ensure that eligible citizens receive timely support from the Rhode Island State Government.

1.4 PROJECT SCOPE:

The scope of the project is defined by the targeted plans, namely SNAP, CCAP, Medicaid, Medicare, and RIW, catering specifically to Rhode Island State citizens. The system encompasses multiple modules, including administration, user management, application registration, data collection, eligibility determination, correspondence, benefit issuance, and reporting. The project's scope extends to the development of a user-friendly interface, robust backend microservices, and seamless integration with external services.

1.5 OBJECTIVES:

The Integrated Eligibility System (IES) aims to address the challenges identified in the requirements and problems statement by focusing on the following objectives:

- **Efficient Plan Management:** The system will develop an administrative module to effectively manage health and insurance plans, as well as case workers. This module will provide flexibility in plan creation and modification, ensuring that the administration can adapt to changing needs and conditions.
- **Automate Application Process:** IES will implement an online application system to streamline the process of applying for health and insurance plans. By eliminating manual paperwork and long queues, the system will reduce the time and effort required for citizens to apply for benefits, thereby improving accessibility and efficiency.
- **User-Friendly Interface:** A key objective is to create an intuitive user interface for citizens, facilitating easy registration, login, and plan application submission.
- **Accurate Eligibility Determination:** The system will implement a robust Eligibility Determination module to accurately assess citizen eligibility for each plan based on predefined conditions. By ensuring accuracy in eligibility determination, IES will minimize errors and inconsistencies, improving the fairness and effectiveness of benefit allocation.
- **Timely Benefit Issuance:** IES will enable the Benefit Issuance module to securely transfer funds to approved citizens on a monthly basis. This seamless integration with external services will ensure that eligible citizens receive their benefits in a timely manner, addressing their financial needs promptly and efficiently.
- **Comprehensive Reporting:** The Reports module will be developed to generate detailed and insightful reports in Excel and PDF formats. These reports will aid in system monitoring and analysis, providing stakeholders with valuable insights into system performance, usage patterns, and areas for improvement.

By focusing on these objectives, the Integrated Eligibility System (IES) will streamline the management of health and insurance plans, improve citizen access to benefits, and enhance overall system efficiency and effectiveness.

CHAPTER 2

LITERATURE REVIEW

2.1 MOTIVATIONS:

Motivations for the Integrated Eligibility System (IES) project stem from a genuine concern for addressing the health and insurance needs of Rhode Island State citizens facing financial constraints. Foremost among these motivations is the imperative to prioritize Citizen Welfare. Through the implementation of IES, the government endeavours to extend timely support to those requiring assistance, thereby elevating their quality of life and fostering social welfare.

Additionally, the project seeks to address issues of Administrative Efficiency inherent in current manual processes for managing health and insurance plans. By automating these processes through IES, we aim to significantly reduce administrative overhead and minimize errors, thereby enhancing the efficiency of plan management, eligibility determination, and benefit issuance.

Furthermore, IES is designed with a focus on Accessibility and Inclusivity, recognizing the barriers faced by many citizens in accessing and navigating the complexities of health and insurance plans. Through a user-friendly interface and streamlined application processes, IES aims to break down these barriers, ensuring equitable access to benefits for all citizens, irrespective of their background or circumstances.

Data Security and Accuracy are paramount considerations in the development of IES, given the risks associated with traditional systems lacking robust security measures. By implementing stringent security protocols and advanced eligibility determination algorithms, IES prioritizes the protection and accuracy of citizen information, ensuring responsible use and safeguarding against unauthorized access, data breaches, and privacy violations.

Moreover, IES emphasizes Systematic Monitoring and Analysis, recognizing the importance of comprehensive reporting and monitoring capabilities in evaluating system performance and effectiveness. Through detailed reports and insights, stakeholders can track application status, monitor eligibility determinations, and assess program outcomes, enabling data-driven decision-making and continuous improvement efforts.

2.2 PROBLEM DESCRIPTION:

The Rhode Island Integrated Eligibility System (RI-IES) faces various challenges in its current state, impacting the efficiency and effectiveness of managing health and insurance plans for citizens with financial constraints. The following problems highlight the shortcomings of the existing system:

- ✓ Inefficient Process:
 - The current system for managing health and insurance plans is manual, time-consuming, and error-prone.
 - Administrative tasks such as application processing, eligibility determination, and benefit issuance require significant time and effort.
- ✓ Limited Accessibility:
 - Citizens face barriers in accessing and applying for health and insurance plans due to complex eligibility criteria, paperwork requirements, and limited digital literacy.
 - Lack of user-friendly interfaces and online application options restricts access for certain demographics.
- ✓ Inaccurate Eligibility Determination:
 - There are instances of inaccuracies in assessing citizen eligibility for various plans, leading to delays or denials of benefits.
 - Manual processes and subjective decision-making contribute to inconsistencies and errors in eligibility determination.
- ✓ Limited Reporting and Monitoring:
 - The current reporting and monitoring capabilities are insufficient for evaluating the performance and effectiveness of the eligibility system.
 - Stakeholders lack access to real-time data and comprehensive reports for tracking application status, monitoring eligibility determinations.
- ✓ Data Security Concerns:
 - Data security measures in the existing system may be inadequate, posing risks of unauthorized access, data breaches, and privacy violations.
 - Citizen information stored in disparate systems may lack robust encryption and access controls, raising concerns about data protection and confidentiality.

RI-IES aims to modernize and integrate systems, automate processes, improve accessibility, enhance accuracy in eligibility determination, centralize data management, and ensure robust data security measures. This optimization effort aims to improve citizen access to benefits, enhance system efficiency, and effectiveness in managing health and insurance plans.

CHAPTER 3

PRELIMINARIES

3.1 METHODOLOGIES USED:

The methodologies used in the development of the Integrated Eligibility System (IES) project primarily revolve around modern software development practices and frameworks. The project adopts an Agile methodology, specifically Scrum, to enable iterative and incremental development, allowing for flexibility and adaptability to changing requirements throughout the project lifecycle.

- **Agile Development Methodology:**

Although Agile is typically used for team-based projects, a single developer could still use some of the Agile principles to manage their project. For example, the developer could break down the project into smaller, manageable tasks and prioritize them based on importance. The developer could also use Agile practices like frequent testing and feedback to ensure the platform is working as expected.

The RI-IES project adopts the Agile development methodology to ensure flexibility and adaptability throughout the development lifecycle. Regular sprints, continuous feedback loops, and iterative development practices are employed to accommodate evolving requirements and deliver a high-quality, user-focused system.

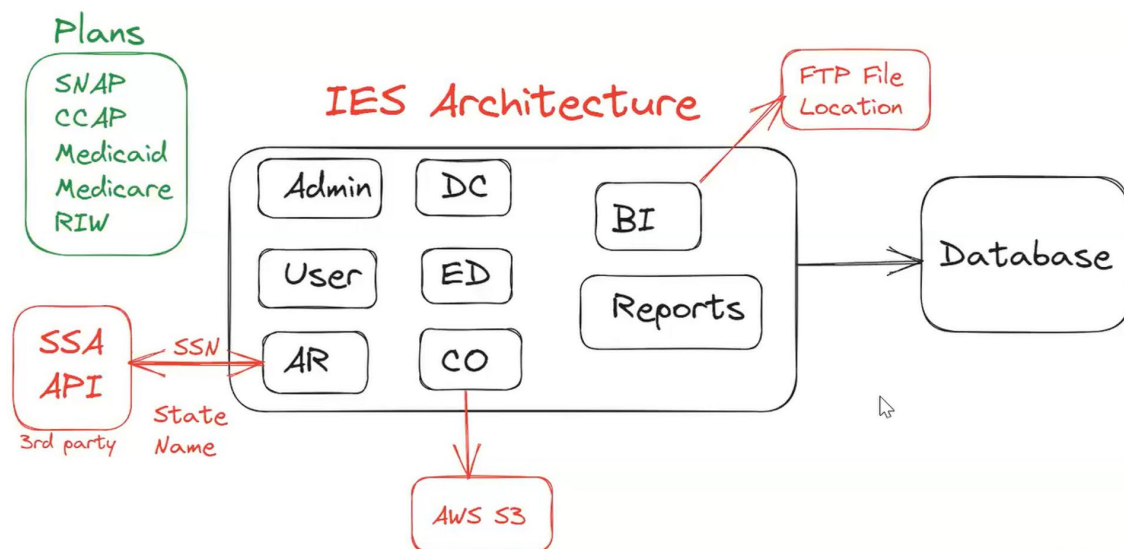
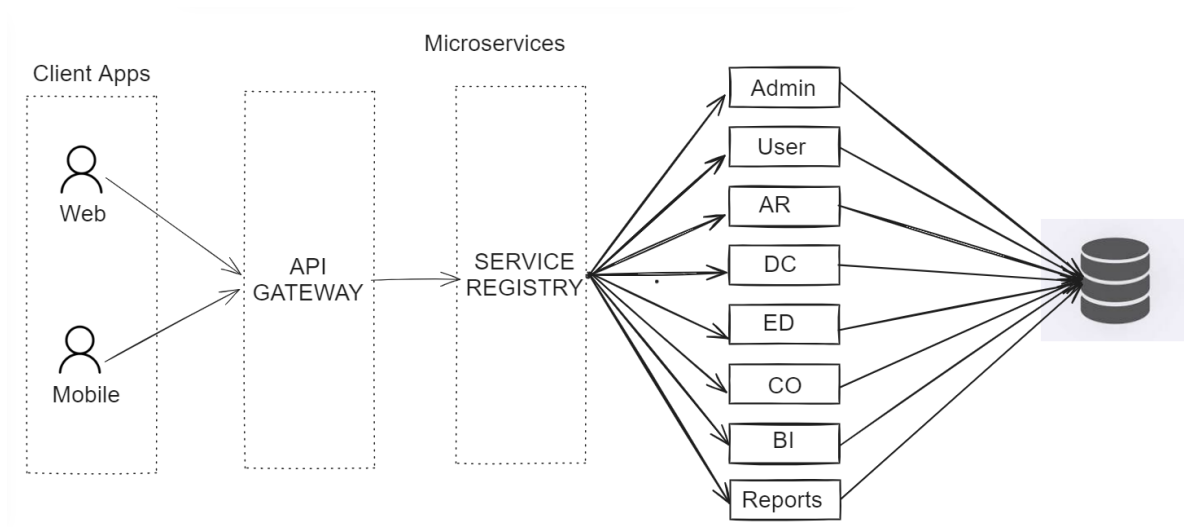
3.2 SYSTEM ARCHITECTURE:

In the Integrated Eligibility System (IES) project, we have adopted a microservices architecture to structure the system, utilizing the Spring Boot framework for development. This architectural approach offers several advantages that align with the project's requirements and objectives:

- **Modularity:** Microservices architecture decomposes the system into smaller, independent services, each responsible for specific business functions or features. This modularity allows for better organization of code and enables teams to work on individual services autonomously, promoting faster development cycles and easier maintenance.

- **Scalability:** With microservices, each service can be scaled independently based on its workload and resource requirements. This scalability ensures that the system can handle varying levels of traffic and workload efficiently, improving performance and responsiveness.
- **Flexibility:** Microservices architecture promotes flexibility and agility in system design and development. New features or changes can be implemented and deployed independently in specific services without affecting the entire system. This decoupling of services minimizes the risk of system-wide failures and allows for faster iteration and innovation.
- **Resilience:** By distributing functionality across multiple services, microservices architecture enhances system resilience and fault tolerance. If one service encounters issues or failures, it does not impact the entire system, as other services can continue to function independently. Additionally, redundant instances of critical services can be deployed to mitigate single points of failure.
- **Technology Diversity:** Microservices architecture enables the use of different technologies and programming languages for individual services based on their specific requirements. This flexibility allows teams to select the most appropriate tools and technologies for each service, optimizing performance, and productivity.
- **Deployment and Updates:** Microservices architecture facilitates continuous deployment and updates by breaking down the system into smaller components. Each service can be deployed independently, enabling faster release cycles and reducing the risk associated with deploying large, monolithic applications.
- **System Expansion:** As the Integrated Eligibility System evolves and grows, microservices architecture supports seamless system expansion. New services can be added or existing services can be modified or replaced without disrupting the entire system, enabling scalability and future-proofing the architecture.

By adopting a microservices architecture in conjunction with the Spring Boot framework, the Integrated Eligibility System is well-equipped to meet the project's objectives of scalability, flexibility, resilience, and maintainability, while facilitating efficient development, deployment, and system expansion.



3.3 TECHNOLOGIES USED:

1. Spring Boot Framework:

Spring Boot is an open-source Java-based framework used for building microservices. It simplifies the development process by providing default configurations, eliminating the need for manual setup, and offering a wide range of features for building enterprise-grade applications. Its lightweight nature and extensive ecosystem provide a solid foundation for developing robust, scalable, and maintainable backend microservices.

We utilized Spring Boot to create multiple standalone microservices, each responsible for specific business functionalities such as user authentication, data processing, and API endpoints. Spring Boot's auto-configuration and dependency injection capabilities allowed us to quickly bootstrap these services without manual setup.

2. Java Programming Language:

Java is a widely-used, platform-independent programming language known for its simplicity, reliability, and versatility. It has a strong community of developers and a mature ecosystem of libraries and frameworks.

Java is chosen as the primary language for implementing backend logic due to its suitability for enterprise-level applications, ease of debugging, and strong support for multithreading and concurrency.

We wrote backend logic using Java, implementing service layers, data access objects (DAOs), and business logic components within each microservice. Java's object-oriented programming paradigm enabled us to create modular and maintainable code, while its rich ecosystem provided libraries for handling tasks such as JSON parsing, encryption, and HTTP communication.

3. MySQL Database:

MySQL is a popular open-source relational database management system (RDBMS) known for its reliability, performance, and scalability. It uses SQL (Structured Query Language) for managing and manipulating data.

MySQL is used for storing structured data such as citizen information, application data, and system records, providing a reliable and efficient storage solution for the project.

We designed and implemented MySQL database schemas to store various types of data, including citizen profiles, application forms, audit logs, and configuration settings. We utilized SQL queries to perform CRUD operations, create indexes for optimizing query performance, and enforce referential integrity through foreign key constraints.

4. RESTful APIs:

Representational State Transfer (REST) is an architectural style for designing networked applications. RESTful APIs use HTTP requests to perform CRUD (Create, Read, Update, Delete) operations on resources, providing a standardized and interoperable interface for communication between microservices.

We designed RESTful APIs using Spring MVC within the Spring Boot microservices, defining resource endpoints, request/response formats, and HTTP methods for interacting with backend services. These APIs enabled seamless communication between frontend and backend components, supporting features such as user authentication, data retrieval, and transaction processing.

5. Angular Framework:

Angular is a TypeScript-based open-source web application framework developed by Google. It is widely used for building single-page applications and dynamic web interfaces.

Angular's component-based architecture and two-way data binding capabilities facilitate the development of responsive, interactive, and maintainable frontend user interfaces.

With Angular, we developed a modern and responsive web application frontend, featuring interactive user interfaces for managing citizen data, submitting applications, and viewing system reports. Angular's component-based architecture facilitated code reuse and modular development, while its powerful data binding capabilities ensured real-time updates of UI components.

3.4 TOOLS USED:

Integrated Development Environment (IDE):

IntelliJ IDEA or Eclipse: Used as the primary IDE for Java development, providing features such as code editing, debugging, version control integration, and project management.

Version Control:

Git: Utilized for version control, enabling collaborative development, code sharing, and tracking changes across multiple developers. Platforms like GitHub, GitLab, or Bitbucket may be used for hosting repositories and managing pull requests.

Build Automation:

Maven or Gradle: Used for dependency management and project build automation, facilitating the management of project dependencies, compilation, testing, and packaging.

Containerization:

Docker: Employed for containerization of microservices, packaging each service and its dependencies into lightweight, portable containers. Docker Compose may be used for managing multi-container Docker applications.

Container Orchestration:

Kubernetes: Utilized for container orchestration, automating deployment, scaling, and management of containerized applications across a cluster of machines. Kubernetes provides features for service discovery, load balancing, and rolling updates.

API Documentation:

Swagger or OpenAPI: Used for documenting RESTful APIs, providing a standardized format for describing API endpoints, request/response formats, and authentication mechanisms. Swagger UI be used to generate interactive API documentation.

Database Management:

MySQL Workbench : Used for database administration, query development, and schema management in MySQL databases.

Project Management:

Jira, Trello, or Asana: Employed for project planning, task tracking, and agile development management, facilitating collaboration and workflow coordination among team members.

Testing Frameworks:

JUnit or TestNG: Used for writing and executing unit tests for Java backend code.

Protractor or Cypress: Utilized for end-to-end testing of Angular frontend applications.

These tools complement the technologies used in your project, providing support for various aspects such as development, testing, deployment, monitoring, and collaboration, thereby enhancing productivity and efficiency throughout the project lifecycle.

CHAPTER 4

METHODS & IMPLEMENTATION

4.1 ACTORS:

- **Admin**
 - Manages plans and case workers.
 - Accesses reports for system monitoring.

- **User**
 - Registers, log in, and applies for plans.
 - Views the dashboard for plan status.

- **Case Worker**
 - Manages citizen applications.
 - Assists in eligibility determination.

4.2 LIFE CYCLE MODEL:

The development life cycle follows the Agile methodology, with iterative sprints, continuous feedback, and frequent releases. This approach allows for flexibility and responsiveness to changing requirements, ensuring the timely delivery of increments.

4.3 FUNCTIONALITIES REQUIREMENTS:

a) Admin Module:

The system enables administrators to manage plan lifecycle, assign case workers, and access detailed reports for effective program oversight and decision-making.

- Plan creation, modification, and deletion.
- Case worker assignment and management.
- Access to comprehensive reports.

◆ Account Controller

➤ R1 : Registration / Create case worker account

Description : The endpoint facilitates the registration process by creating a new case worker account in the Integrated Eligibility System (IES) based on the provided user account form data.

Inputs:

- Full name
- email address
- mobile phone
- gender
- dob
- ssn
- activeSw

➤ R2 : Change account status / UnlockUserAccount.

Description : The endpoint facilitates the unlocking of a user account by updating its status and password based on the provided user ID and unlock form data.

Inputs:

- email address
- tempPwd (getting from Registration throw email)
- newPwd

confirmPwd

➤ R3 : deleteUser Acc

Description : Admin can delete case worker account permanently

Inputs:

- User id

◆ User Controller

➤ R1 : login

Description : The login endpoint authenticates the user's credentials provided in the loginForm, allowing access to the system's features. However, prior execution of unlockUserAccount is necessary to enable login if the user's account is locked.

Inputs:

- Email id

Password

➤ R2 : Checking Dashboard status.

Description : User can check their dashboard status by providing email id

Inputs:

- Email id

◆ Plan Controller

➤ R1 : CreatePlan

Description : This method is responsible for creating a new plan based on the information provided in the PlanForm object. It takes the plan category, plan name, start date, and end date as input parameters.

Inputs:

- Plan Category
- Plan Name
- Plan Start Date
- Plan End Date

➤ R2 : changePlanStatus

Description : This method is used to change the status of a plan identified by its ID. It takes the plan ID and the new status as input parameters.

Inputs:

- Plan id
- Plan status (Active / Inactive)

➤ R3 : Fetch All Plans

Description : This endpoint retrieves all the plans available in the system

➤ R4 : Delete Plan

Description : The endpoint deletes the plan identified by the provided planId

Inputs:

- PlanId

b) Application Registration (AR) Module:

The Application Registration (AR) module captures citizen application details and validates the provided data before storing it securely in the system.

- Capture citizen application details.
- Validate and store application data.

➤ R1: Create Application

Description : The createApplication service within the Application Registration (AR) module facilitates the submission of citizen application details. Internally, it interacts with the SSA API (Social Security Administration) to authenticate and verify the citizenship status of the applicant.

Inputs:

- citizen name
- citizen Email
- citizen phone no
- citizen gender
- citizen dob
- citizen ssn

c) Data Collection (DC) Module:

In the Data Collection (DC) Module, the system collects comprehensive information regarding citizen plans, including details such as income, education, and children's data. This collected data is then stored and managed to ensure its availability for eligibility determination processes, facilitating efficient assessment and decision-making regarding citizens' eligibility for various programs or services.

- Collect detailed citizen plan information.
- Ensure availability of data for eligibility determination.

➤ R1: Plan Selection

Description : This Process represents the information related to a citizen's plan selection, including the case number, plan ID, and user ID associated with the selection. This data is crucial for linking specific plans to individual cases and users within the system.

Inputs:

- Case number
- Plan id
- User id

➤ **R2: Income Data Collect**

Description : This process defines the structure for collecting income-related data for a citizen, including their case number, user ID, and various sources of income such as salary, rent, and property income. This information is essential for assessing the financial status of citizens and determining their eligibility for assistance or benefits.

Inputs:

- Case number
- User id
- Salary income
- Rent income
- Property income

➤ **R3: Education Data Collect**

Description : This process facilitates the collection of education-related data for a citizen, including their case number, user ID, highest degree attained, graduation year, and the name of the university attended. This information is crucial for evaluating a citizen's educational background, which may influence their eligibility for certain programs or benefits.

Inputs:

- Case number
- User id
- Highest degree
- Graduation year
- University name

➤ **R4: Kids Data Collect**

Description : This process is responsible for gathering information about the children of a citizen, including a list of kid representing each child, along with the citizen's case number and user ID. This data is essential for assessing the family composition and dependencies, which can impact eligibility for various social welfare programs or benefits.

Inputs:

- List of kids
- Case number
- User id

➤ R5 : Data Collection Summary

Description : This endpoint retrieves a summary of data collected during the data collection process for a specific citizen's case identified by the provided case number. This process likely contains aggregated information from various modules within the Data Collection (DC) module, such as income, education, and kids' data. This summary can provide insights into the citizen's circumstances and help determine their eligibility for relevant programs or services.

Inputs:

- Case number

d) Eligibility Determination (ED) Module:

The Eligibility Determination (ED) Module verifies citizen eligibility based on predefined criteria associated with specific plans or programs, and subsequently approves or denies applications accordingly.

- Verify citizen eligibility based on plan conditions.
- Approve or deny applications accordingly.

➤ R1 : Determine Eligibility

Description : To determine eligibility, you would typically retrieve the necessary information associated with the given case number, evaluate it against the eligibility criteria defined for the relevant plans or programs, and then make a decision whether to approve or deny the application. This process involves checking various factors such as income, education, family composition, and other relevant details to assess eligibility.

Inputs:

- Case number

➤ R2: Generate Notice

Description : To generate a Certificate of Eligibility (Co) for a specific case number, you would gather the necessary information related to the applicant's eligibility status and generate a Co

document accordingly. This document serves as official proof of the applicant's eligibility for the specified plan or program.

Inputs:

- Case number

e) Correspondence (CO) Module:

In the Correspondence (CO) Module, the system generates and sends notices to citizens regarding their application status or any other relevant communication. These notices serve to inform citizens about the progress of their application, approval, denial, or any other updates related to their case.

- Generate and send notices to citizens.
- Communicate application status.

➤ R2: Generate Notice

Description : In this method, the system performs several actions including generating the notice, uploading it to an S3 bucket, updating the notice URL in the database, and finally sending the notice via email as an attachment. These actions collectively ensure that the notice is created, stored securely, and communicated to the intended recipients effectively.

Inputs:

- Notice id

➤ R2: Getting notices by status

Description : In this method, notices are retrieved based on their status. This allows users to query the system for notices with specific statuses, enabling efficient management and tracking of correspondence activities.

Inputs:

- Status (Pending / History)

f) Benefit Issuance (BI) Module:

In the **BI** (Benefit Issuance) module, benefits are distributed to eligible citizens following approval. This involves transferring benefit amounts to approved individuals and interfacing with external services for fund transfers to ensure timely and accurate disbursement of benefits.

- Issue benefit amounts to approved citizens.
- Interface with external services for fund transfer.

g) Reports Module:

In the **Reports** module, various types of reports are generated, including plan-wise, status-wise, and monthly reports. These reports are exported in Excel and PDF formats to provide comprehensive insights and analysis for stakeholders.

- Generate plan-wise, status-wise, and monthly reports.
- Export reports in Excel and PDF formats.

4.4 NON-FUNCTIONALITIES & REQUIREMENTS:

There are a lot of software requirements which are included in the non-functional requirements in this project, which includes Security, Performance, Usability, Scalability, Availability, Reliability, Compatibility and Accessibility.

1. **Security:** The portal should ensure the confidentiality, integrity, and availability of user data, as well as protect against unauthorized access or modification.
2. **Performance:** The portal should be able to handle a large number of concurrent users and requests, while maintaining a fast response time.
3. **Usability:** The portal should be easy to use, navigate, and understand for both students and teachers, with clear instructions and feedback.
4. **Scalability:** The portal should be able to scale up or down based on changing user needs, without compromising performance or functionality.
5. **Availability:** The portal should be available 24/7, with minimal downtime for maintenance or upgrades.
6. **Reliability:** The portal should be reliable and free from errors or bugs, with automated testing and monitoring to detect and fix issues.
7. **Compatibility:** The portal should be compatible with a wide range of browsers, devices, and operating systems, to ensure that users can access it from anywhere and on any device.
8. **Accessibility:** The portal should be accessible to users with disabilities, with features such as text-to-speech and keyboard navigation.

These non-functional requirements are important for ensuring the success and usability of the Assignment Portal, and should be considered throughout the development process

4.5 PRIORITY TABLE:

Admin Module Functionality Table:

Functionality	Priority	Description
Plan Management	High	Create, modify, and delete health and insurance plans.
Case Worker Management	High	Assign, track, and manage case workers responsible for processing applications.
Access to Comprehensive Reports	High	View and generate detailed reports on plan applications, eligibility determinations, etc.
System Configuration	Medium	Configure system settings, such as eligibility criteria and notification templates.
Data Management	Medium	Manage data related to citizens, applications, plans, and benefits.
Security Administration	Medium	Administer user access, roles, permissions, and ensure data security measures are in place.

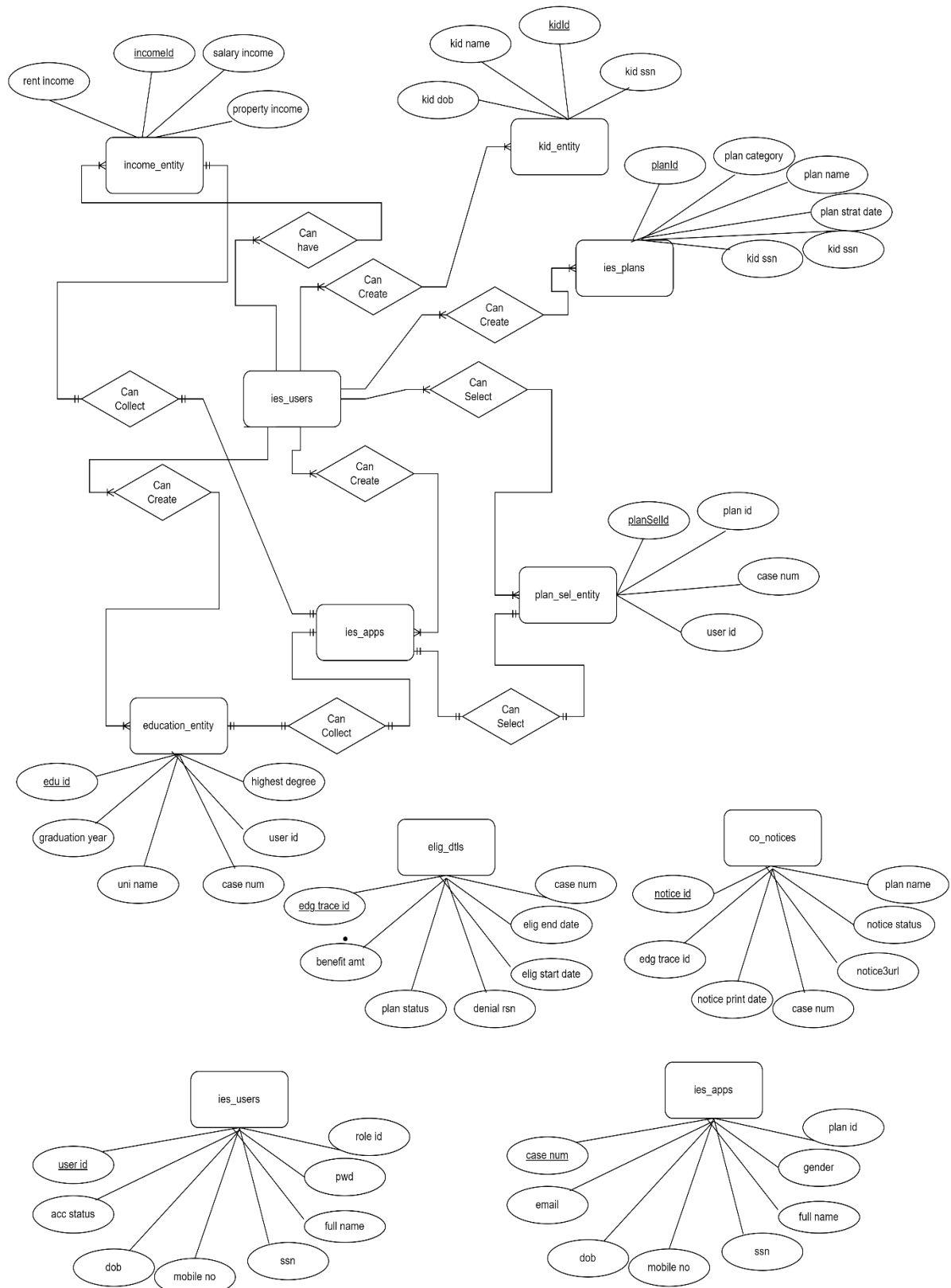
User Module Functionality Table:

Functionality	Priority	Description
User Registration	High	Register new users and authenticate existing ones.
Plan Application Submission	High	Submit applications for health and insurance plans.
Dashboard for Plan Status Tracking	High	Track the status of submitted applications and view related updates.
Account Management	Medium	Update user profile information, preferences, and account settings.
Document Upload	Medium	Upload required documents for plan application and verification.
Support and Helpdesk	Medium	Access user support services, FAQs, and assistance channels.

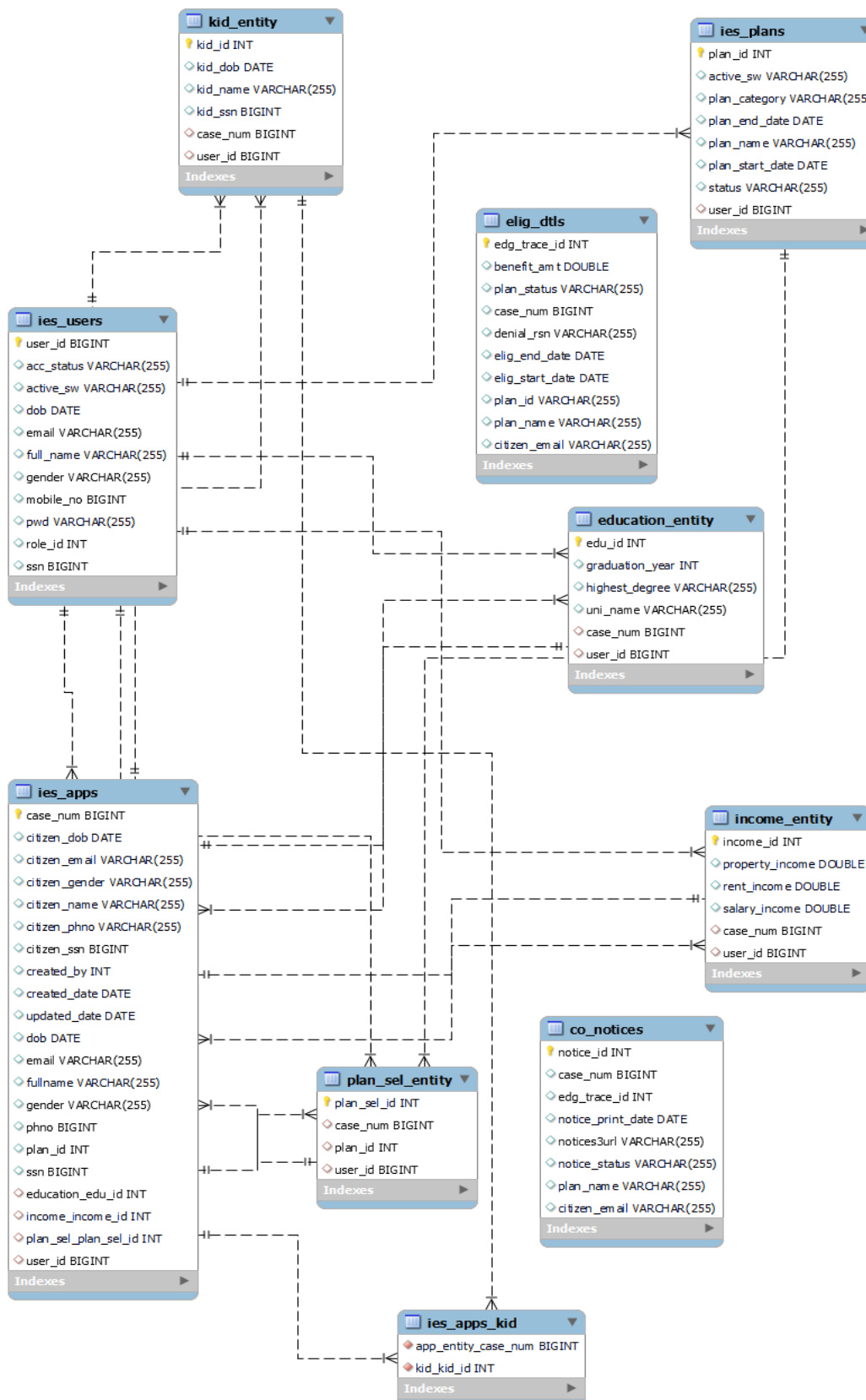
Caseworker Module Functionality Table:

Functionality	Priority	Description
Case Management	High	Manage and process applications, including eligibility determination and benefit issuance.
Citizen Interaction	High	Communicate with citizens regarding application status, requirements, and follow-ups.
Application Details and Status	High	Access and update application details, review eligibility status, and track application progress.
Report Generation	Medium	Generate reports on caseloads, application status, eligibility determinations, etc.
Training and Resources	Medium	Access training materials, guidelines, and resources for efficient case handling.
Collaboration and Communication	Medium	Collaborate with other caseworkers, supervisors, and external stakeholders as needed.

4.6 ENTITY RELATIONSHIP DIAGRAM:



4.7 DATABASE SCHEMA DIAGRAM:



4.8 DATA FLOW DIAGRAM:

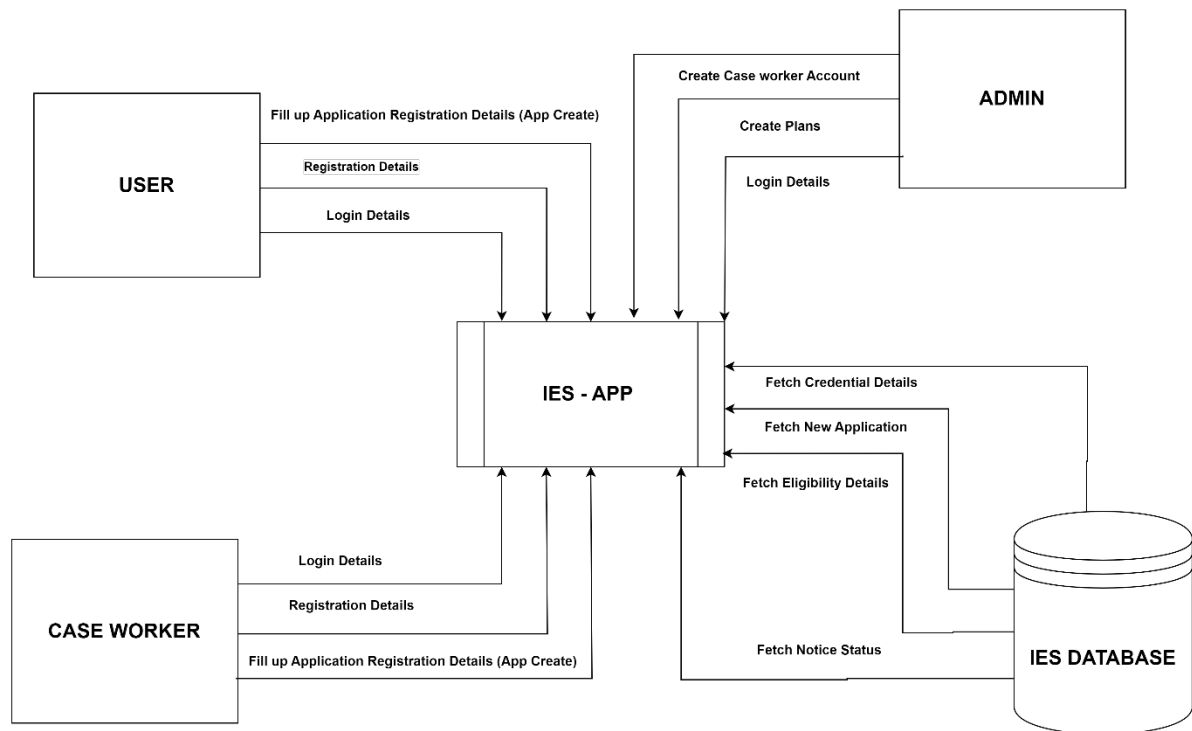


Figure 4.8.1 (DFD – Context Level)

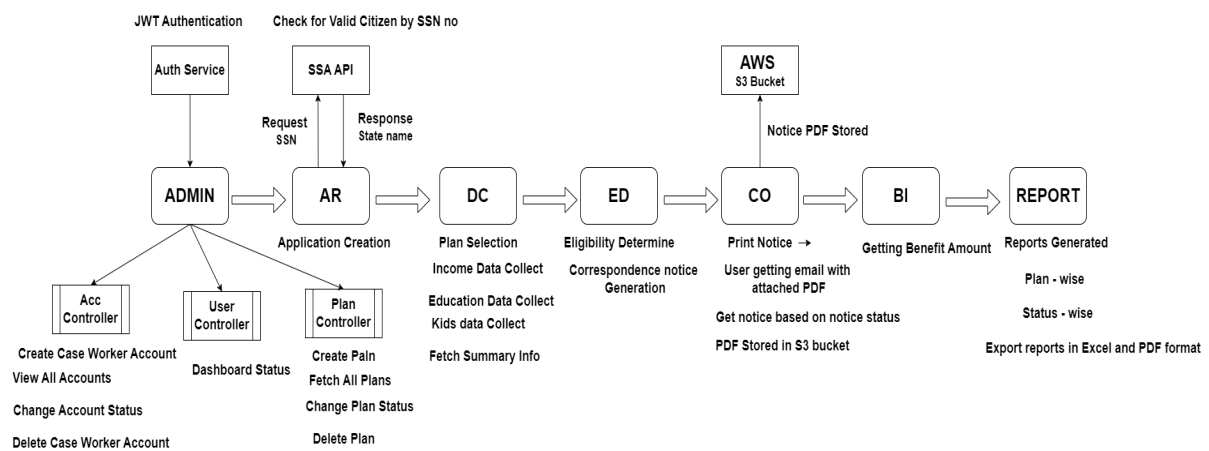


Figure 4.8.2 (DFD – Level 1)

4.9 CODE SNIPPET:

- **User Entity** : This will create the User entity table at the Persistence (Here MySQL used)

```
5  import java.time.LocalDate;
6  import java.util.List;
7
8  @Entity
9  @Table(name="IES_USERS")
10 public class UserEntity {
11
12     @Id
13     @GeneratedValue(strategy = GenerationType.IDENTITY)
14     @Column(name = "user_id")
15     private Long userId;
16     private String fullName;
17     private String email;
18     private String pwd;
19     private Long mobileNo;
20     private String gender;
21     private LocalDate dob;
22     private Long ssn;
23     private String accStatus;
24     private String activeSw;
25
26     private Integer roleId;
27
28     @OneToMany(mappedBy = "user", cascade=CascadeType.ALL)
29     private List<PlanEntity> plans;
30
31 }
```

Figure 4.9.1 (User Entity)

- **Plan Entity** : This will create the Plan entity table at the Persistence (Here MySQL used) , which is responsible for creating the plans by Admin.

```
32 import jakarta.persistence.*;
33 import java.time.LocalDate;
34
35 @Entity
36 @Data
37 @Table(name="IES_PLANS")
38 public class PlanEntity {
39
40     @Id
41     @GeneratedValue(strategy = GenerationType.IDENTITY)
42     private Integer planId;
43     private String planCategory;
44     private String planName;
45     private LocalDate planStartDate;
46     private LocalDate planEndDate;
47     private String activeSw;
48
49     @Column(name = "status")
50     private String status;
51
52     @ManyToOne(cascade = CascadeType.ALL)
53     @JoinColumn(name="user_id")
54     private UserEntity user;
55 }
```

Figure 4.9.2 (Plan Entity)

- **Eligibility Determination Entity** : This will create the eligibility determination entity table at the Persistence (Here MySQL used) , which is responsible for determining the eligibility for the plan based on condition.

```

56 import java.time.LocalDate;
57 import jakarta.persistence.Entity;
58 import jakarta.persistence.GeneratedValue;
59 import jakarta.persistence.GenerationType;
60 import jakarta.persistence.Id;
61
62 import jakarta.persistence.Table;
63
64 @Entity
65 @Table(name = "ELIG_DTLS")
66 public class EligEntity {
67
68     @Id
69     @GeneratedValue(strategy = GenerationType.IDENTITY)
70     private Integer edgTraceId;
71
72     private String planId;
73
74     private Long caseNum;
75
76     private String planStatus;
77
78     private Double benefitAmt;
79
80     private LocalDate eligStartDate;
81
82     private LocalDate eligEndDate;
83
84     private String denialRsn;
85
86     private String planName;
87
88     private String citizenEmail;
89 }

```

Figure 4.9.3 (Plan Entity)

- **Plan Rest Controller :** This code is related to the creation of case worker account by admin.

```

90 import java.util.List;
91
92 @RestController
93 @RequestMapping("/admin")
94 public class AccountRestController {
95
96     @Autowired
97     private AccountService_accService;
98
99     @PostMapping("/user")
100     public ResponseEntity<String> createAccount(@RequestBody_UserAccForm
userAccForm) {
101         logger.debug("Account Creation Process Started...");
102
103         try {
104             SignUpResponse_status =accService.createUserAccount(userAccForm);
105             logger.debug("Account Creation Process Completed...");
106
107             if (status != null && status.getSuccessMsg() != null) {
108                 logger.info("Account Created Successfully.");
109                 return ResponseEntity.status(HttpStatus.CREATED)
110                     .body(status.getSuccessMsg());
111             } else {
112                 logger.error("Account Creation Failed: " +
status.getErrMsg());
113                 return
ResponseEntity.status(HttpStatus.INTERNAL_SERVER_ERROR)
114                     .body(status.getErrMsg());
115             }
116         } catch (Exception e) {
117             logger.error("Exception occurred during account creation: ", e);
118             return ResponseEntity.status(HttpStatus.INTERNAL_SERVER_ERROR)
119                 .body("Exception occurred during account
creation");
120         }
121     }
122 }
123

```

Figure 4.9.4 (Plan Rest Controller)

CHAPTER 5

RESULT AND OUTPUTS

- **Account Create :-** this is the output of account creation of case worker

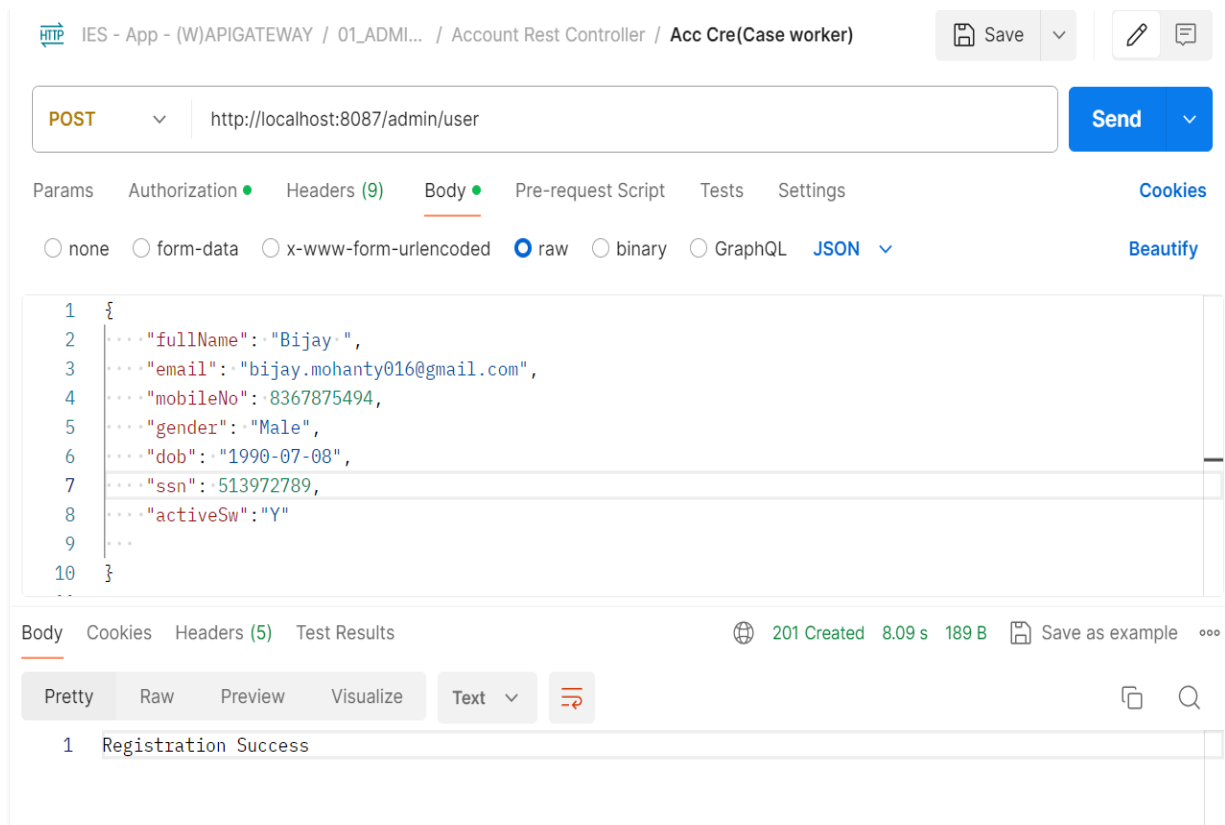


Figure 5.1 (Account Creation for case worker)

- **Email** :- when account create immediately case worker got email

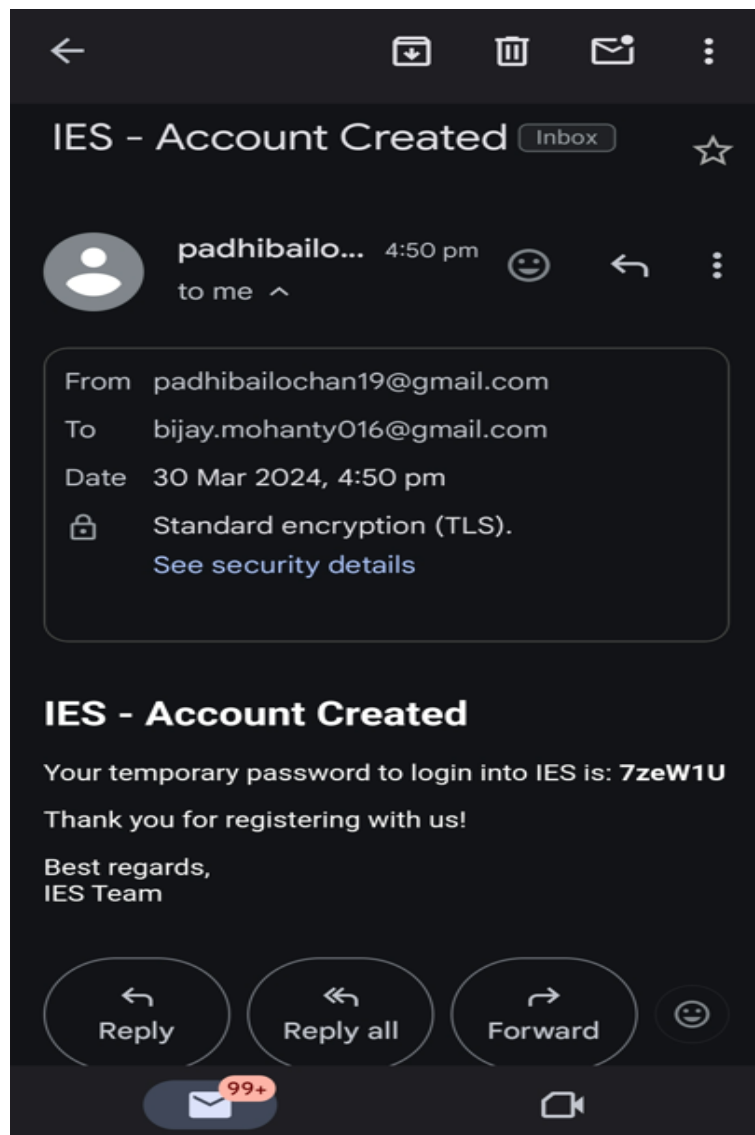


Figure 5.2 (Email generated for case worker)

- **Application Create :-** this is the output of Application creation of user

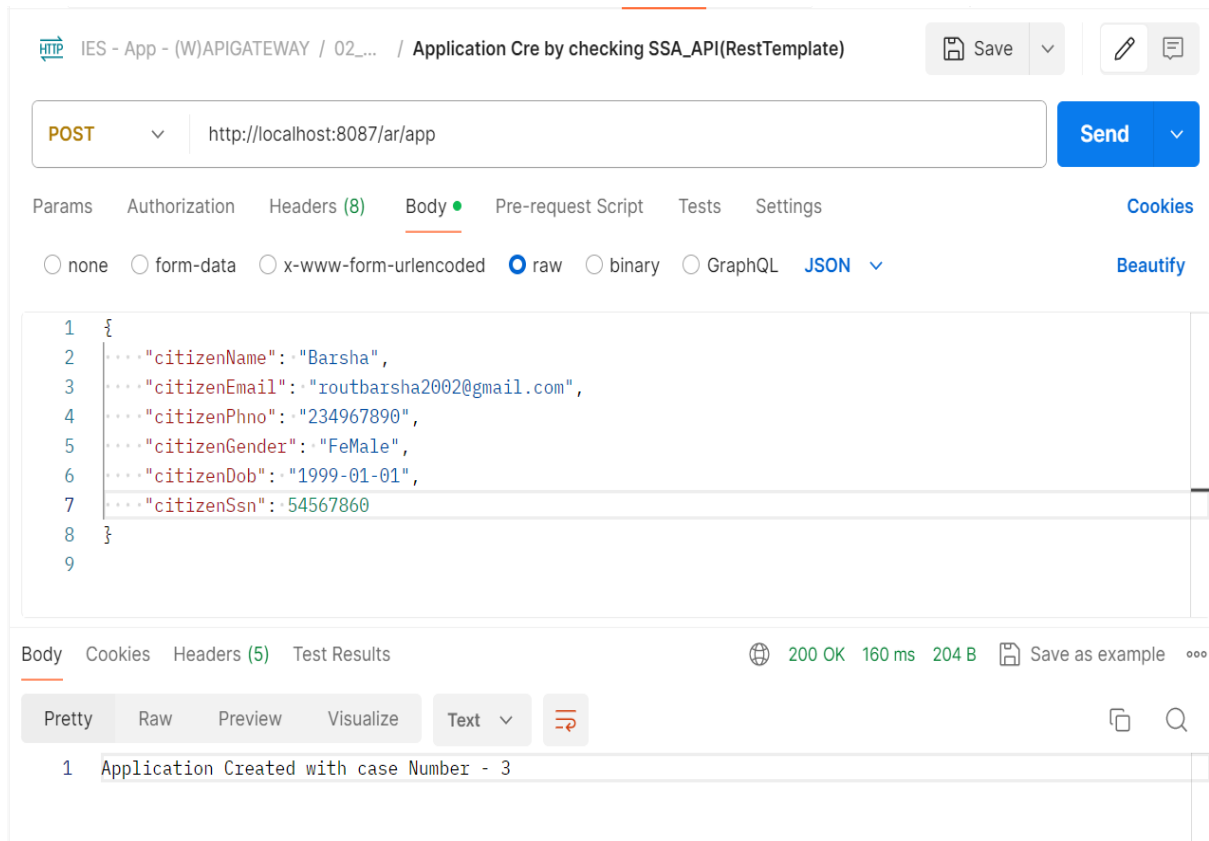


Figure 5.3 (Application Creation for user)

- **Eligibility Determine by IES App :-** this is the output of the user application that he/she is eligible for applied plan or not .

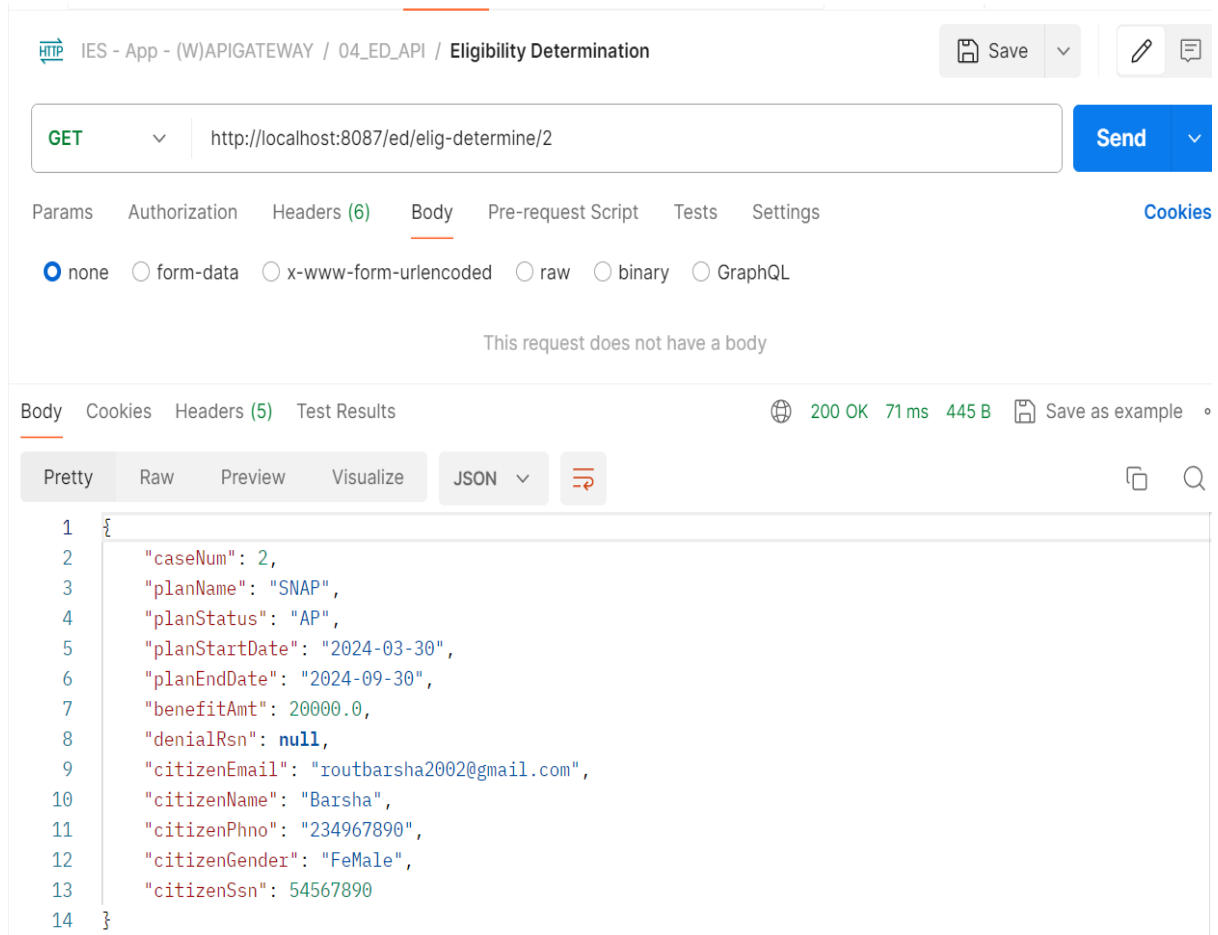


Figure 5.4 (Eligibility Determination)

CHAPTER 6

CONCLUSION AND FUTURE WORK

The RI-IES delivers a robust system performance, an intuitive user interface, and efficient plan application, eligibility determination, correspondence, benefit issuance, and reporting processes. Through systematic testing and user feedback, the system demonstrates its effectiveness in meeting the objectives outlined in the project documentation. Ongoing monitoring and feedback mechanisms are in place to ensure continuous improvement and adaptation to evolving user needs.